

2 주차 학습 노트

2022-16684 재료공학부 안휘준

과제 1. diff 로 개선된 소스 비교

과제에서 알려 준 diff 명령으로 Q.09e 의 두 파일을 이전 단계(Q.09c, Q.09d)의 파일과 나란히 놓고 비교했다.

```
diff -y -W 128 ../Q.09c_Enhance_Keyboard_Stim/sc_invaders_TB.cpp ./sc_invaders_TB.cpp | less
diff -y -W 128 ../Q.09d_Enhance_GLCD_TLM/sc_glcd128x64_TLM.cpp ./sc_glcd128x64_TLM.cpp | less
```



그림 1. sc_invaders_TB.cpp 비교 (왼쪽 Q.09c, 오른쪽 Q.09e). 가운데에 | 표시가 있는 줄이 서로 다른 줄이다.

다른 줄이 많아서 처음에는 코드가 크게 바뀐 줄 알았다. 그런데 하나씩 보니 대부분 //로 시작하는 주석(코드에 붙인 설명 글)이었다. 그래서 주석과 띄어쓰기 차이를 빼고 코드만 다시 비교했다. 그러자 두 파일 모두 화면에 출력하는 안내 문장 몇 줄만 달랐고, 실제로 동작하는 코드는 같았다.

파일	주석을 빼고 비교했을 때 다른 부분	판단
sc_invaders_TB.cpp (Q.09c 와 비교)	안내 문장 1 줄	기능 변화 없음
sc_glcd128x64_TLM.cpp (Q.09d 와 비교)	안내 문장 3 줄, 띄어쓰기 1 곳	기능 변화 없음

그렇다면 기능은 언제 들어왔는지 궁금해서, 처음 단계인 Q.09b 와도 비교했다. 기능은 Q.09c 와 Q.09d 에서 추가됐고, Q.09e 는 두 단계를 합치고 주석과 문장을 다듬은 버전이었다.

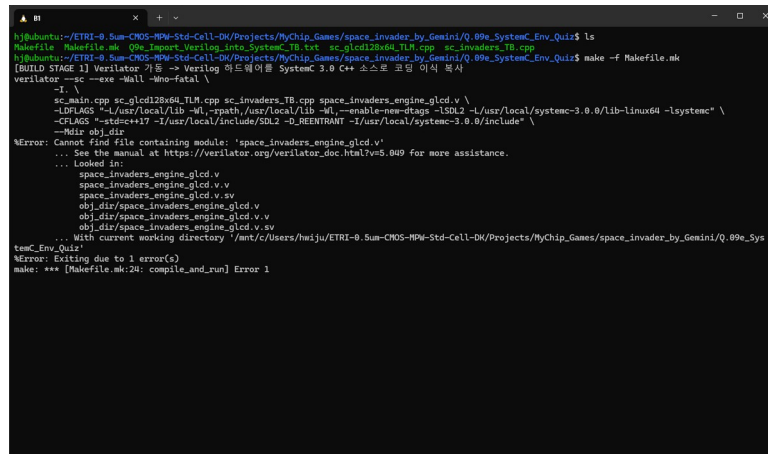
추가된 기능	들어온 단계
키보드 입력을 항상 최신 상태로 읽기	Q.09c
창을 옮겨도 화면이 멈추지 않게 처리	Q.09d
창을 닫으면 시뮬레이션도 같이 종료	Q.09d
그래픽 가속이 안 되는 PC 에서도 화면 표시	Q.09d

과제 2. Makefile.mk 고치기

Gemini 가 만든 Makefile.mk(빌드 방법을 적어 둔 파일)는 그대로 실행하면 빌드되지 않았다. 오류 하나를 고치면 다음 오류가 나와서, 다섯 번 고친 뒤에야 실행에 성공했다.

순서	오류 내용	원인	고친 방법
1	필요한 파일을 찾을 수 없음	파일이 Q.09e 가 아니라 Q.09b 폴더에 있음	Q.09b 폴더 경로를 적어 줌
2	만들어진 파일 이름이 다름	Makefile 에 적힌 이름이 실제로 만들어지는 이름과 다름	이름을 맞게 고침
3	헤더 파일을 찾을 수 없음	컴파일이 다른 폴더 안에서 실행돼서 경로가 어긋남	전체 경로로 적어 줌
4	신호 연결 오류	자동으로 만들어진 신호와 기존 코드의 신호 종류가 다름	Verilator 옵션 하나 추가
5	빌드는 됐지만 실행이 안 됨	SystemC 설치 폴더가 내 PC 와 다름	내 PC 의 설치 폴더로 고침

2-1. 첫 번째 오류: 파일이 없다



```
h@ubuntu:~/ETRI-0_Sum-CHOS-MPM-Std-Cell-DK/Projects/MyChip_Games/space_invader_by_Gemini/Q_09e_SystemC_Env_Quiz$ ls
Makefile Makefile.mk Q09e_Import_Verilog_IRts_SystemC_TB.txt sc_glcd128x64_TLM.cpp sc_invaders_TB.cpp
h@ubuntu:~/ETRI-0_Sum-CHOS-MPM-Std-Cell-DK/Projects/MyChip_Games/space_invader_by_Gemini/Q_09e_SystemC_Env_Quiz$ make -f Makefile.mk
[BUILD STAGE 1] Verilator 가동 => Verilog 하드웨어를 SystemC 3.0 C++ 소스로 코딩 이식 복사
verilator --sc --no-wall --no-fatal \
  -I. \
  -sc_main.cpp sc_glcd128x64_TLM.cpp sc_invaders_TB.cpp space_invaders_engine_glcd.v \
  -LDFLAGS "-L/usr/local/lib -Wl,-rpath,/usr/local/lib -Wl,-enable-new-dtags -lsdl2 -L/usr/local/systemc-3.0.0/lib-linux64 -lsystemc" \
  -CFLAGS "-std=c++17 -I/usr/local/include/sdl2 -D_REENTRANT -I/usr/local/systemc-3.0.0/include" \
  --dir obj_dir
%Error: Cannot find file containing module: 'space_invaders_engine_glcd.v'
... See the manual at https://verilator.org/verilator_doc.html?v=5.049 for more assistance.
... Looked in:
  space_invaders_engine_glcd.v
  space_invaders_engine_glcd.v.v
  space_invaders_engine_glcd.v.v.v
  obj_dir/space_invaders_engine_glcd.v
  obj_dir/space_invaders_engine_glcd.v.v
  obj_dir/space_invaders_engine_glcd.v.v.v
  ... With current working directory '/mnt/c/Users/hwiju/ETRI-0_Sum-CHOS-MPM-Std-Cell-DK/Projects/MyChip_Games/space_invader_by_Gemini/Q_09e_Sys
temC_Env_Quiz'
%Error: Exiting due to 1 error(s)
make: *** [Makefile.mk:24: compile_and_run] Error 1
```

그림 2. 처음 실행한 결과. 필요한 파일을 현재 폴더에서 찾지 못한다.

Q.09e 폴더에는 .cpp 파일 두 개밖에 없었다. 폴더를 찾아보니 필요한 파일 4 개가 모두 Q.09b 폴더에 있어서, Makefile 에 그 폴더 경로를 적어 주었다. 파일을 복사해 오는 방법도 있지만, 같은 파일이 두 곳에 생기지 않도록 경로만 지정했다.

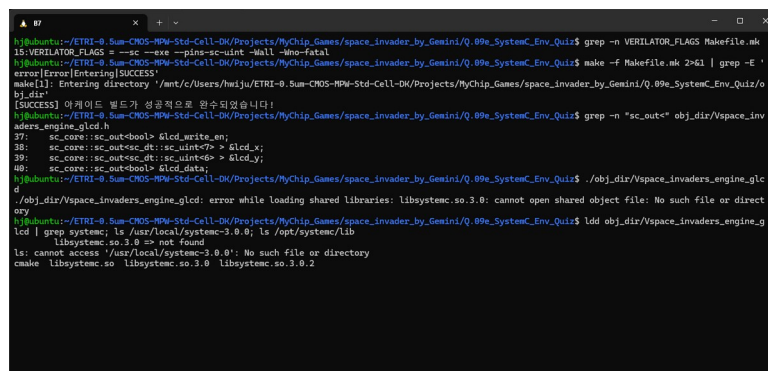
2-2. 두 번째~네 번째 오류

두 번째 오류는 Makefile 에 적힌 파일 이름이 실제로 만들어진 파일 이름과 달라서 생겼다. 이름을 맞게 고치니 해결됐다.

세 번째 오류(헤더 파일을 찾을 수 없음)에서는 조금 헤맸다. Q.09b 폴더를 "현재 위치에서 한 칸 위"라는 뜻의 ../Q.09b 로 적었는데도 계속 찾지 못했다. 로그를 다시 읽어 보니 컴파일어 obj_dir 이라는 하위 폴더 안에서 실행되고 있었다. 기존 위치가 한 칸 아래로 바뀌어서 경로가 어긋난 것이었다. 처음부터 전체 경로를 적어 주니 해결됐다.

네 번째 오류는 Verilator 가 자동으로 만든 신호와 기존 코드의 신호 종류(숫자를 담는 형식)가 달라서 생겼다. 기존 코드를 고치는 대신 Verilator 에 옵션을 하나 추가해서 신호 종류를 맞췄다.

2-3. 다섯 번째 오류: 빌드는 됐는데 실행이 안 된다



```
h@ubuntu:~/ETRI-0_Sum-CHOS-MPM-Std-Cell-DK/Projects/MyChip_Games/space_invader_by_Gemini/Q_09e_SystemC_Env_Quiz$ grep -n VERILATOR_FLAGS Makefile.mk
15:VERILATOR_FLAGS = --sc --exe --pins-sc-uint --wall --no-fatal
h@ubuntu:~/ETRI-0_Sum-CHOS-MPM-Std-Cell-DK/Projects/MyChip_Games/space_invader_by_Gemini/Q_09e_SystemC_Env_Quiz$ make -f Makefile.mk 2>&1 | grep -E '
error[Error]Entering[SUCCESS]
make[1]: Entering directory '/mnt/c/Users/hwiju/ETRI-0_Sum-CHOS-MPM-Std-Cell-DK/Projects/MyChip_Games/space_invader_by_Gemini/Q_09e_SystemC_Env_Quiz/o
bj_dir'
[SUCCESS] 아래이드 빌드가 성공적으로 완수되었습니다!
h@ubuntu:~/ETRI-0_Sum-CHOS-MPM-Std-Cell-DK/Projects/MyChip_Games/space_invader_by_Gemini/Q_09e_SystemC_Env_Quiz$ grep -n "sc_out<" obj_dir/vspace_inv
aders_engine_glcd.h
37:   sc_core::sc_out<bool> glcd_write_en;
38:   sc_core::sc_out<sc_dt::sc_uint<7>> glcd_x;
39:   sc_core::sc_out<sc_dt::sc_uint<6>> glcd_y;
40:   sc_core::sc_out<bool> glcd_data;
h@ubuntu:~/ETRI-0_Sum-CHOS-MPM-Std-Cell-DK/Projects/MyChip_Games/space_invader_by_Gemini/Q_09e_SystemC_Env_Quiz$ ./obj_dir/vspace_invaders_engine_glc
d
./obj_dir/vspace_invaders_engine_glcd: error while loading shared libraries: libsystemc.so.3.0: cannot open shared object file: No such file or direct
ory
h@ubuntu:~/ETRI-0_Sum-CHOS-MPM-Std-Cell-DK/Projects/MyChip_Games/space_invader_by_Gemini/Q_09e_SystemC_Env_Quiz$ ldd obj_dir/vspace_invaders_engine_g
lcd | grep systemc; ls /usr/local/systemc-3.0.0; ls /opt/systemc/lib
libsystemc.so.3.0 => not found
ls: cannot access '/usr/local/systemc-3.0.0': No such file or directory
ls: cannot access 'libsystemc.so.3.0 libsystemc.so.3.0.2'
```

그림 3. 빌드 성공([SUCCESS]) 메시지가 나왔지만, 실행하면 SystemC 라이브러리를 찾지 못한다.

빌드 성공 메시지가 나와서 끝난 줄 알았는데, 실행하니 SystemC 라이브러리를 찾지 못했다. Makefile 에 적힌 SystemC 설치 폴더가 내 PC 에는 없었다. 빌드할 때는 Verilator 가 설치될 때 기억해 둔 경로를 대신 써서 통과했고, 실행할 때 문제가 드러났다. 경로를 내 PC 에 맞게 /opt/systemc 로 고치니 실행됐다.

2-4. 실행 확인

창을 띄우니 처음에는 체크무늬만 보여서 화면이 깨진 줄 알았다. Verilog 코드를 보니 시작 대기 화면에서는 일부러 체크무늬(테스트 화면)를 보여 주도록 되어 있었다. Space 를 누르면 게임이 시작됐고, 방향키와 Space 로 이동과 발사가 잘 됐다. 창을 닫으면 시뮬레이션도 같이 종료됐다.

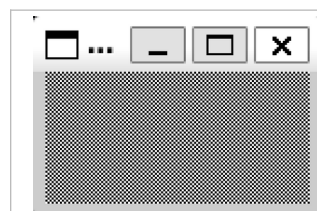


그림 4. 시작 대기 화면

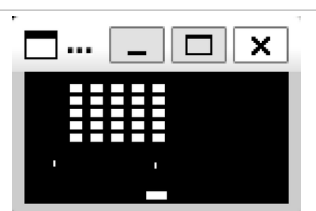
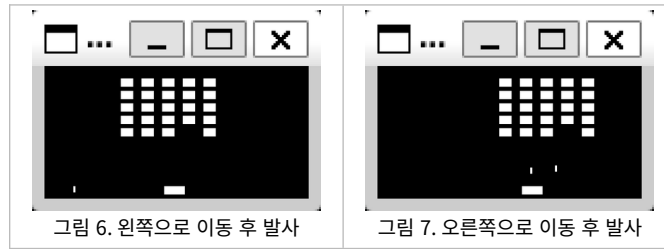


그림 5. Space 를 누르면 게임 시작



과제 1, 2 느낀 점

- 주석만 보면 코드가 크게 바뀐 것 같았지만, 실제로는 거의 그대로였다. AI는 버전마다 주석을 새로 쓰는 경우가 많아서, 설명보다 코드를 직접 확인해야 한다는 것을 알았다.
- 오류는 한 번에 하나씩만 나타났다. 하나를 고치면 다음 오류가 나와서, 오류 메시지를 끝까지 읽고 하나씩 고쳐 나가는 수밖에 없었다.
- 빌드가 성공했다고 끝난 것이 아니었다. 실제로 실행해 봐야 숨어 있던 문제까지 찾을 수 있었다.
- 코드 자체는 이미 잘 동작하는 상태였고, 문제는 대부분 파일 위치나 설치 폴더처럼 내 PC 환경과 관련된 것이었다. AI는 내 PC 상황을 모르기 때문에, 이런 부분은 사람이 직접 확인해야 한다.

과제 3. 내 신경망 만들기 (3x8 디코더)

블로그 "[내 신경망 만들기/2 부] 2. 파이썬으로 작성하는 신경망"의 마지막 실습을 따라 했다. 입력 3 개, 중간(은닉) 노드 20 개, 출력 8 개인 작은 신경망에게 3x8 디코더를 가르치는 실습이다. 예를 들어 입력이 011 이면 3 번 출력만 0.99 에 가깝게 나와야 한다. 신경망 코드는 블로그에 있는 것을 그대로 썼다.

3-1. 학습 횟수 늘리기

블로그에서는 50 번 학습한 뒤 111 을 넣었을 때 정답인 7 번 출력과 오답인 3 번 출력이 둘 다 0.19 정도로 나와서 구분하지 못했다. 그래서 학습 횟수를 늘려 보았다. 학습 함수가 매번 입력과 목표를 화면에 출력해서 여러 번 반복하면 화면이 가득 찼기 때문에, 출력하는 두 줄은 막아 두었다. 또 실행할 때마다 결과가 조금씩 달라서(처음 가중치를 무작위로 정하기 때문), 무작위 값을 고정해 놓고 학습 횟수만 바꿔 가며 비교했다.

학습 횟수	8 개 중 맞힌 개수	111 입력: 7 번 출력(정답)	111 입력: 나머지 중 최대
1	2	0.300	0.383
10	4	0.166	0.135
50	8	0.337	0.189
100	8	0.595	0.146
200	8	0.781	0.076
500	8	0.892	0.042
1000	8	0.931	0.029
5000	8	0.973	0.008

표. 학습 횟수에 따른 결과. 목표값은 0.99 이다.

50 번일 때 8 개를 모두 맞추기는 했지만, 111 의 정답 출력이 0.34 로 낮고 오답(0.19)과 차이가 크지 않았다. 100 번부터 차이가 벌어졌고, 500 번이면 정답 출력이 0.9 가까이 올라갔다. 5000 번까지 늘려도 0.97 정도에서 더 오르지 않았다. 무작위 값을 바꿔 20 번 다시 해 보니, 50 번 학습에서는 한 번 틀렸고 100 번 학습에서는 20 번 모두 맞혔다.

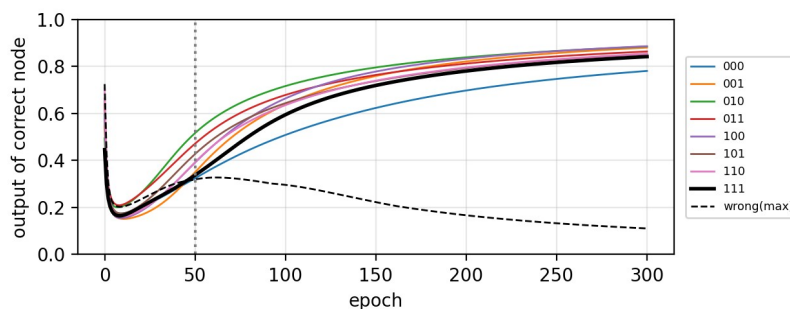


그림 8. 학습 횟수에 따른 8 개 입력의 정답 출력. 검은 점선은 틀린 출력 중 가장 큰 값, 세로 점선은 블로그의 50 번.

그래프로 그려 보니 예상과 달랐다. 처음 10 번 정도는 정답 출력이 오히려 0.2 아래로 떨어졌다가 그 뒤에 올라갔다. 목표값 8 개 중 7 개가 0 이라서, 처음에는 모두 작게 내보내는 쪽으로 먼저 바뀌는 것 같다. 정답 출력이 점선을 넘어서는 곳이 50 번 근처였다. 블로그의 50 번 결과가 애매했던 이유가 이것으로 보인다.

또 000 이 가장 늦게 올라왔다. 입력이 모두 0 이면 가중치를 곱해도 0 이 되기 때문에, 입력 쪽 가중치를 아무리 바꿔도 결과가 달라지지 않아서인 것 같다.

3-2. 가중치 변화 그려 보기

학습할 때마다 중간 노드에서 출력으로 가는 가중치(8x20 개)를 저장해 두고, 나중에 그래프로 그렸다.

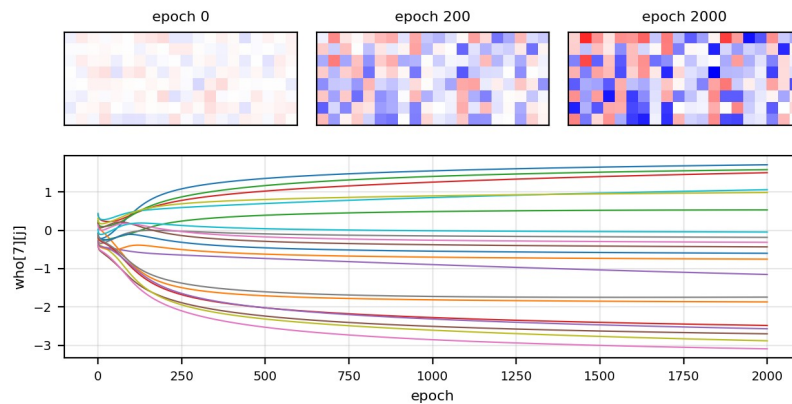


그림 9. 위: 가중치를 색으로 나타낸 것 (빨강 +, 파랑 -). 아래: 7 번 출력(111 담당)으로 들어오는 가중치 20 개의 변화.

처음에는 연한 색이 무작위로 흩어져 있었는데, 학습할수록 색이 진해졌다. 즉 가중치 값이 점점 커졌다(처음 -0.5~0.6, 2000 번 뒤 -3.6~3.0). 아래 그래프를 보면 가중치들이 처음 250 번 정도 동안 크게 바뀌면서 +쪽과 -쪽으로 갈라지고, 1000 번이 넘으면 거의 변하지 않는다.

출력이 목표에 가까워지면 오차가 작아지고, 1 부에서 본 시그모이드 그래프도 0 이나 1 근처에서는 평평하다. 그래서 가중치를 고치는 양도 점점 작아지는 것으로 이해했다. 3-1 에서 5000 번을 돌려도 0.97 에서 멈춘 것도 같은 이유 같다.

3-3. 과제 3 에서 느낀 점

- 학습 횟수만 50 에서 500 으로 바꿨는데 결과가 크게 달라졌다. 애매하던 111 이 확실하게 맞았다. 신경망은 코드만큼 학습을 얼마나 시키느냐도 중요하다는 것을 알았다.
- 실행할 때마다 결과가 달라서 처음에는 뭔가 잘못된 줄 알았다. 가중치를 무작위로 정하고 시작하기 때문이었다. 1 부에서 말한 "임의로 정한 초기값"이 이런 뜻이라는 것을 직접 보았다.
- 숫자만 볼 때는 무슨 일이 일어나는지 잘 몰랐는데, 그래프로 그려 보니 처음에 떨어졌다가 올라가는 것이 한눈에 보였다.
- 3x8 디코더는 논리회로에서 배운 AND 게이트 8 개로 바로 만들 수 있다. 그런데 신경망은 가중치 220 개를 수백 번 고쳐야 겨우 비슷하게 따라 했다. 답을 이미 아는 문제는 회로로 만드는 편이 낫고, 신경망은 손글씨처럼 규칙을 정하기 어려운 문제에 쓰는 것이라는 말이 이해됐다.